Fall Detection by 1D-CNN based on Public Dataset Using a Wrist-worn Smartwatch

Songsheng Li
Hong Kong College of Technology
Hong Kong, China
Songsheng.li@hotmail.com

***Abstract*— Falls among the elderly population pose a significant threat, often leading to severe injuries, loss of independence, and even fatality. To address this critical issue, researchers have been exploring various approaches to develop effective fall detection systems. In this study, we present a novel fall detection algorithm that leverages the WEDA-FALL dataset, a comprehensive dataset that includes data from wrist-worn sensors capturing both simulated and real-world fall incidents involving elderly individuals. The key insight behind our approach is the observation that falls often exhibit distinct patterns in sensor data. To capture these patterns, we employ a one-dimensional convolutional neural network (1D-CNN) architecture, which has been shown to be well-suited for time-series data analysis. By training our model on the WEDA-FALL dataset, we were able to achieve an impressive accuracy of 94% in fall detection, surpassing the commonly reported accuracy levels of around 90% in the existing literature. The high performance of our algorithm can be attributed to the use of the WEDA-FALL dataset and the effectiveness of the 1D-CNN architecture in extracting and learning the characteristic patterns in the sensor data. Notably, our algorithm offers the key features of being quick, highly accurate, and cost-effective, making it a promising solution for practical deployment in real-world settings to improve the safety and well-being of the aging population.**

Keywords— Fall Detection, Accelerometer, 1D-CNN, WEDA-FALL

I. INTRODUCTION

Falls are a serious concern for individuals of all ages, but they pose a particular risk to older adults. The consequences of a fall can range from minor injuries to life-threatening complications. The benefits of successful fall detection: (1) reducing risk of serious injury as fall detection systems can alert caregivers or emergency services immediately, (2) maintaining independence because fall detection empowers individuals to remain active and engaged in their daily lives without fear of falling. (3) peace of mind for caregivers by knowing that a loved one is protected by fall detection technology, and (4) early intervention and prevention. In conclusion, fall detection is an essential tool for promoting safety, independence, and overall well-being, especially for older adults.

Fall detection algorithms are the computational brains behind the technology that safeguards individuals from the consequences of falls. These algorithms process data collected from various sensors to accurately identify when a fall has occurred. Broadly, fall detection algorithms can be categorized into three primary approaches based on data sources.

A. Wearable Sensor-Based Algorithms

These algorithms utilize data from sensors embedded in wearable devices such as smartwatches, accelerometers, gyroscopes, and heart rate monitors. There are 3 popular methods: (1) threshold-based methods, paper [1] and [2] establish predefined experimental thresholds for accelerometers values from wrist-worn watch. When these thresholds are met during a specific time frame, a fall is detected. (2) Machine learning-based methods. Saleh et al. [3] employ algorithms like Support Vector Machines (SVM), Random Forests, or Naive Bayes, and deep learning methods to learn patterns from their proposed dataset, FallAllD to classify a fall. They applied deep learning for higher accuracy. (3) Deep learning-based methods. Authors [4] leverage CNN to find that a sampling rate of 20 Hz can maximize the effectiveness of a fall detector, leading to high accuracy in fall detection.

B. Camera-Based Algorithms

These algorithms analyse video footage to detect falls. One way is body shape change analysis, Espinosa et al. [5] analyzes images in a section of time, extracts features using an optical flow method which can obtain information on the relative motion between two consecutive images to judge the change then conclude fall or not. The other popular way is 3D skeleton model, focusing on the rapid and uncontrolled movement of the body during a fall, employing 3D skeleton points given by a tracking camera, the dataset [6] can help accurately detect fall events. The visual ways trigger the concern of privacy.

C. Ambient Sensor-Based Algorithms

These algorithms utilize sensors placed in the environment, such as pressure sensors, microphones, or radar-based systems.

- Pressure sensor-based methods: Using floor vibration as the recognition source, the system [7] incorporates a person identification through vibration produced by footsteps to identify who is the fallen person, and the system can distinguish fall from other ADL, and reach an accuracy of 95%.
- Acoustic-based methods: Analyze sound patterns for unusual noises associated with a fall. The system [8] collect normal acoustic signals (footstep sound signals), then Mel-frequency cepstral coefficient (MFCC) features are extracted from the processed signals and used to construct a data which will be trained by SVM, a binary result of fall or not will be obtained by the algorithm.
- Radar-based methods: Use radar signals to track human movement and detect abnormal motion patterns. Han et al. [9] collected signal data containing the motion information of objects using an impulse-radio ultra-wideband (IR-UWB) radar, then fed the data to a CNN algorithm to classify “Fall” or “ADL.”

It's important to note that the choice of algorithm depends on factors such as desired accuracy, computational resources, user preferences, and the specific environment where the system will be deployed. Additionally, many systems

combine multiple algorithm types to improve overall performance and robustness.

Developing robust fall detection algorithms requires access to high-quality datasets that accurately represent real-world scenarios, key considerations for fall detection datasets include: (1) sensors such as accelerometer, gyroscope, and magnetometer; (2) fall scenarios, such as forward falls, backward falls, and side falls; (3) Activities of Daily Living (ADLs), and (4) subject diversity. Several public datasets have been made available to facilitate research in this area. FallAllD [3] collected data from three data loggers worn on the waist, wrist, and neck of subjects using accelerometers, gyroscopes, magnetometers, and barometers, the dataset separates fall to 35 conditions and ADLs to 42 types. The other datasets UP-Fall [10], which attained data from multi-sensors include wearable sensors, ambient sensors, and vision equipment, but subjects are young and healthy individuals, so the data is limited in representativeness. Wrist Elderly Daily Activity and Fall Dataset (WEDA-FALL) [11] applied a smartwatch on the wrist and employed elderly people with thorough security measures, it covers 8 types of Falls and 11 Types of ADLs.

The paper proposes an algorithm that utilizes a 1D-CNN based on the WEDA-FALL dataset, as WEDA-FALL collects data from wrist-worn devices and includes a diverse group of participants. The following section will detail the application of 1D-CNN, while Section III will present and analyze the results. Finally, Section IV will conclude with insights and suggestions for further research.

II. Methods

In this section we will start with the dataset, although WEDA-FALL is ideal comparing to other datasets, the diversity of movements make some data very similar to fall, such as D05, 'attempt to get up and collapse into a chair', so training data for 1D-CNN should be chosen carefully, then how the data are fed to the algorithm is elaborated.

A. *Fall pattern*

First, the typical fall pattern is presented in Figure 1. Roughly the process of fall can be divided into 4 stages, which are pre-fall, fall, ground, and rebound. Data of Figure 1 is extracted from 10kz/F01/U01, according to explanation from authors, it is a young man "Fall forward while walking caused by a slip", the data resolution is 10Hz, 10 times a second.

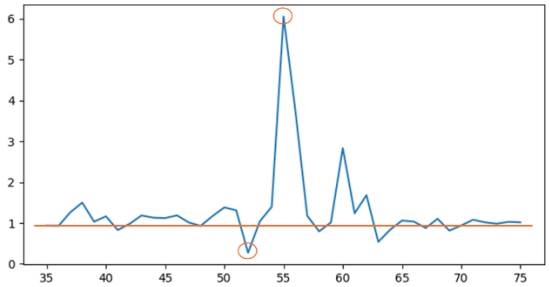

Fig. 1. Pattern of fall.

The x-axis is time, unit of it is 1/10 second and y-axis is combined acceleration (CA) of 3-axis, unit is *g* which is Gravity acceleration. The red line in Figure 1 is the standard gravity acceleration, when people are walking normally or standing still the CA is around *g*. The red circle at the bottom of Figure 1 is the moment of falling, the person is in a state of near weightlessness, then touching ground in the upper red circle, the greater the acceleration, the heavier the fall, which is followed by several rebound.

B. *CNN*

Convolutional neural networks (CNNs) are a type of deep learning architecture particularly well-suited for processing and analyzing spatial data, such as images. CNNs consist of multiple layers that perform convolution, pooling, and fully connected operations to extract relevant features from the input data. The full picture is shown in Figure 2. The convolutional layers apply filters to local regions of the input, allowing the network to learn spatial patterns. The pooling layers then reduce the spatial dimensions of the feature maps, while the fully connected layers perform classification or regression tasks based on the extracted features.

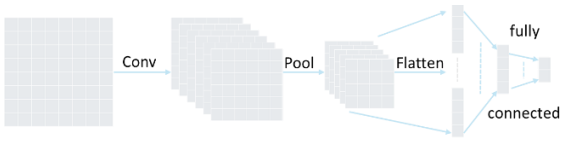

Fig. 2. Principle of CNN.

For the convolution operation, two key hyperparameters are number of filters and kernel size. The number of filters (*f*) determines the depth of the output feature map and the kernel size (*k, k*) defines how much of the input image is being looked at during each convolution operation.

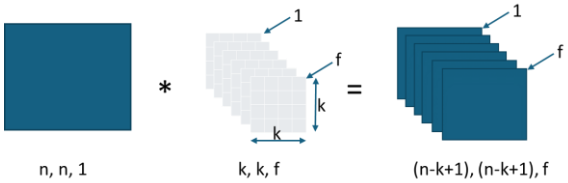

Fig. 3. Convolution operation.

It is not suitable for our 1D-data, but the way it extracts features is effective, so 1D-CNN is introduced.

C. *1D-CNN*

For time-series data like sensor readings, a 1D-CNN architecture is more appropriate, as it operates on 1-dimensional sequences rather than 2D images. 1D-CNNs are able to effectively capture the temporal patterns in sensor data, making them a suitable choice for the fall detection task at hand, where the sensor readings exhibit characteristic patterns associated with fall events. The principle of 1D-CNN is shown in Figure 4.

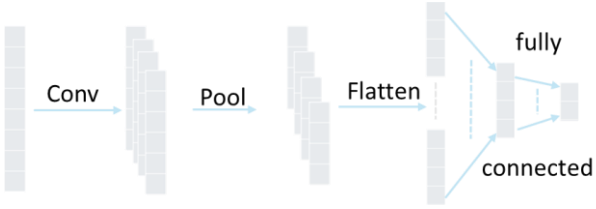

Fig. 4. Principle of 1D-CNN.

1D-CNN will be employed to train and test fall data from WEDA-FALL.

D. Dataset selection

WEDA-FALL is composed of two groups, Falls and ADLs, mentioned before, D05 in ADLs, which is "Sitting a moment, attempt to get up and collapse into a chair", sounds similar to fall, the flowing Figure 5 is five samples from D05, from which, we can see that the 1st, 3rd and 4th are very similar to real fall.

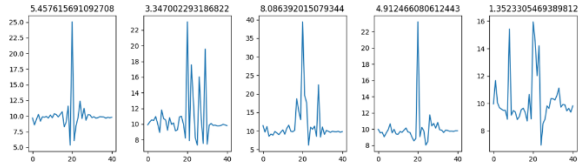

Fig. 5. Samples from D05.

When we choose data for 1D-CNN, data should be labeled for the algorithm, according to the authors of WEDA-FALL, D05 belongs to ADLs, should be labelled as 0 (false, not fall), as some data look like falls, the whole D05 is exclusive, they are not used in this project.

E. Data Preparation

WEDA-FALL is originally captured at 50Hz from a smartwatch on the wrist, the data last about 5-20 seconds each in general, then they are resampled evenly to 5Hz-50Hz. From Figure 1 we can see that typical fall lasts about 2 seconds, so data of 10Hz are used in this project, and only 4 seconds data centered on maximum acceleration are used, by this way, the whole process of fall is included, and computing power is saved because the algorithm only handles small parts of the data. Figure 6 are 5 samples from fall group, F01, the touch ground acceleration is in the middle of the data and the whole process of fall are reserved and clear in the figure.

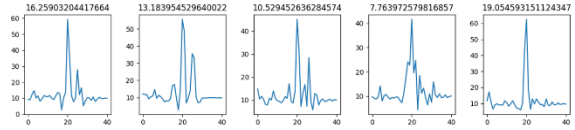

Fig. 6. Samples from F01.

But some data is not typical, for example, F08/U14_R03_accel.csv, which is data from F08 ("Lateral fall while sitting, caused by fainting or falling asleep"), is shown in Figure 7. The maximum acceleration of the dataset is not the touching down action, so this type of data is given up, totally there are 7 files with the same style.

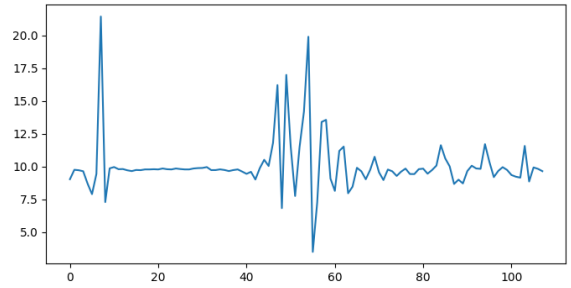

Fig. 7. Abnomal Fall data in Fall.

Finally, totally 343 files of Fall meet the standard and are labelled as 1(true fall).

Similar situation happens to ADLs (not fall), such as D01/U02_R02_accel.csv, which is from ADLs group 1 (walking), is shown in Figure 8. There is not 2 seconds data before the maximum acceleration, so this file is given up too.

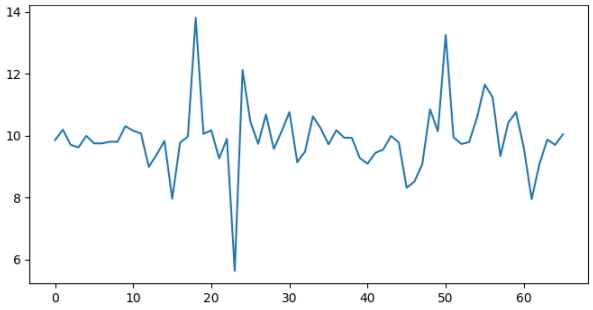

Fig. 8. Abnomal data in ADLs.

Finally, totally 373 files meet the requirements for ADL (not Fall), they are labelled as 0. So total qualified files for our project is 343+373=716, although numbers of Fall and ADL are not absolutely balanced, it is feasible for the algorithm.

III. Results

The main task of this project can be describe as employing 716 labeled datasets to train and test a fall detection model by 1D-CNN algorithm. As 2 seconds data before and after maximum acceleration are used, and sample rate is 10Hz, so totally 2*10*2+1=41 numbers in every file.

The algorithm is implemented on TensorFlow [12] and Scikit-learn [13], the flow of 1D-CNN is shown in Figure 9. First of all, 716 files are divided into training and testing set by general 8:2 mode randomly.

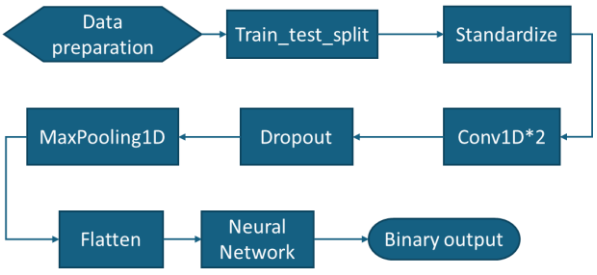

Fig. 9. Flow of 1D-CNN.

The next step is Standardization, which scales the dataset to mean of zero and standard deviation of one.

A. Standardization

The importance of Standardization is incontrovertible, at first thought, as all the data in this case is between 0 to 10*g*, the impact of scale could be ignored, but this should be verified, 10 epochs are tried, the results are shown in Table I.

TABLE I. WITH/WITHOUT STANDARDIZATION

accuracy	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*
without	90.9	88.2	90.3	84.7	90.3	88.9	86.8	88.2	87.5	88.2
with	93.8	94.4	93.8	93	93.8	93.7	93.8	92.4	94.4	92.4

The average accuracy of without Standardization is:

88.403% (+/-1.758)

The average accuracy of with Standardization is:

93.542% (+/-0.698)

The boxplot of those results is compared in Figure 10.

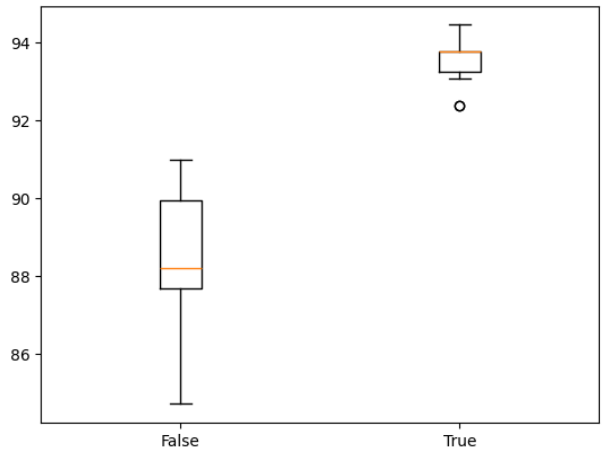

Fig. 10. Boxplot of with/without standardization.

The improvement of accuracy after standardization is distinct as standardization ensures that all features contribute equally. Then the real CNN starts, Conv1D applies convolution operations over the one-dimensional inputs through filters that capture local patterns. The input shape is (41,1) as every input is a one-dimension with 41 items. The filters are key factors in this step, two related hyperparameters are crucial, the first one is number of filters.

B. *Number of filters*

The number of filters in a convolutional layer is a crucial hyperparameter that affects the model's ability to learn and generalize from the data. Six values from 6 to 256 are tried in this case. The results are shown in Table II and Figure 11. It turns out that 64 is the best choice of number of filters.

TABLE II. ACCURACY ON NUMBER OF FILTERS

Number of filters	***8***	***16***	***32***	***64***	***128***	***256***
Accuracy	89.0	91.7	92.6	93.6	92.9	92.9

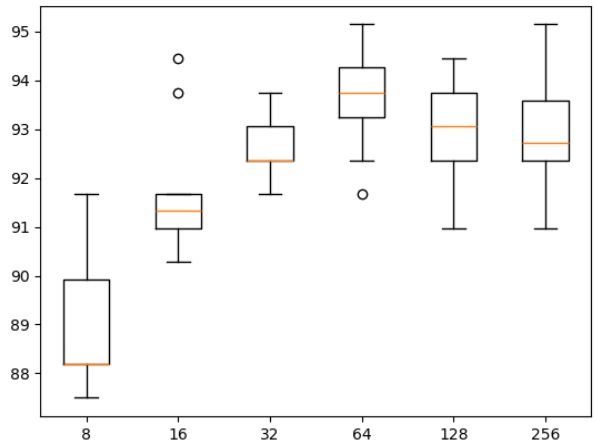

Fig. 11. Boxplot of different number of filters.

While increasing the number of filters can enhance the model's capacity to detect complex patterns, it also raises the computational cost and the risk of overfitting. The results in Table II are aligned with theory, the middle number 64 is a better choice, it balances these factors to build effective convolutional neural networks.

C. *Kernel size*

Kernel size (or filter size) refers to the dimensions of the filter (or kernel) applied during the convolution operation in CNNs. For 1D-CNN, it is a length. Smaller kernels are typically better for capturing local features, while larger kernels can capture more extensive patterns. Five values, 2, 3, 5, 7, and 9 are tried in this case, the results are shown in Table III and Figure 12.

TABLE III. ACCURACY ON KERNEL SIZE

Kernel size	***2***	***3***	***5***	***7***	***11***
Accuracy	92.3	93.1	93.0	93.4	92.2

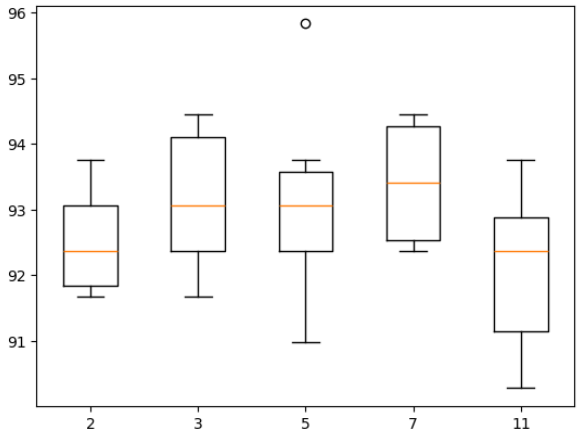

Fig. 12. Boxplot of different kernel sizes.

The choice of kernel size should be balanced with the model's complexity, computational resources, and the specific characteristics of the data. For 1D-CNN, 2 is obvious too small, and 11 is too big to get enough details. The results of 3, 5, and 7 are similar, 7 is a relative better choice.

Stacking multiple convolutional layers increases the capacity of the model to learn more complex representations. This allows the network to understand intricate patterns in the data, so 2 Conv1D are employed to get deeper feature of the data, and avoid the complexity of computation.

Dropout is a regularization technique used in neural networks to prevent overfitting. During each training iteration, a specified percentage of neurons (units) are randomly "dropped out" or deactivated, their output is ignored, this force the model to learn more generalized features. 50% of dropout is set in the training section in our algorithm to ensure that the model does not become too reliant on any particular neuron.

With standardization, number of filters in 64, and kernel size of 7, the designed 1D-CNN can attain an accuracy of about 94%.

D. *Comparison with other algorithms*

Authors of WEDA-FALL do not just provide the original data, they also feed the data to various Machine Learning algorithms, such as kNN (with Euclidean distance, varying the number of neighbors from 1 to 10), SVM and Decision

Tree (DT). The comparison of their results and 1D-CNN is shown in Table IV.

TABLE IV. ACCURACY ON ALGORITHMS

Algorithm	*3NN*	*10NN*	*SVM*	*DT*	*1D-CNN*
Accuracy	94.3	92.5	92.0	88.6	94.0

It is more intuitive in Figure 13.

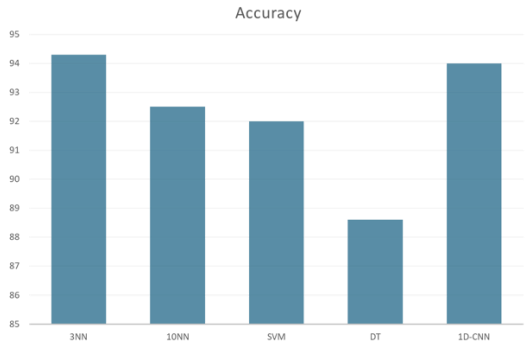

Fig. 13. Accuracy of various algorithms.

The accuracy of proposed algorithm 1D-CNN is very close to the best experimental results from the original dataset creator.

IV. CONCLUSIONS

An algorithm employed 1D-CNN for Fall detection based on public dataset from wrist-worn smartwatch is proposed and tested in this paper. The fall pattern comprises four distinct stages that define the progression of a fall event. These stages are crucial features that the algorithm will extract from the data to identify fall signals effectively. The original dataset consists of 350 fall signals (in 8 groups) and 462 ADL signals (in 11 groups). However, the D05 pattern from the ADL signals was found to be very similar to the fall pattern, so it was excluded. Additionally, some other data files were eliminated as they did not meet the standard requirement of having 2 seconds of data before and after the maximum acceleration. After these exclusions, the final dataset comprised 716 files, including 343 fall signals and 373 ADL signals. In the processing of the 1D-CNN, the 716 files were randomly split into training and testing datasets in an 8:2 ratio. The data was then scaled to ensure that all features contributed equally. Various combinations of filter counts and kernel sizes were tested, and it was determined that 64 filters with a kernel size of 7 provided the best performance. With a 50% dropout rate applied, the algorithm achieved an accuracy of 94%, which is close to the best results reported by the original dataset creator.

With the 1D-CNN achieving such high accuracy, the next step is to implement the algorithm on real equipment. This requires at least 4 seconds of data—2 seconds before a fall and 2 seconds afterward. Fortunately, a lightweight version of TensorFlow is already available for smartwatches, and the performance of this 1D-CNN on real devices will be explored in future studies.

As mentioned earlier, some data files were excluded because they did not meet the requirement of having 2 seconds of data before and after the maximum acceleration. While this standard is essential for identifying falls, it may not be as suitable for ADLs. If the data from ADLs can be processed more effectively, the algorithm's accuracy could be enhanced by leveraging a more comprehensive dataset.

Our team is focused on smart home technologies, particularly in nursing homes, to enhance the well-being of the elderly through innovative IoT solutions. We have conducted research in various related areas [14-15], and with the development of more advanced fall detection algorithms, we aim to integrate this functionality into our evolving smart home health system as a significant step toward achieving our goal.

REFERENCES

[1] S.-L. Hsieh, C.-C. Chen, S.-H. Wu, and T.-W. Yue, “A wrist -worn fall detection system using accelerometers and gyroscopes,” in *Proceedings of the 11th IEEE International Conference on Networking, Sensing and Control*, IEEE, Apr. 2014, pp. 518–523.

[2] S. Li, "Fall Detection With Wrist-Worn Watch by Observations in Statistics of Acceleration," in IEEE Access, vol. 11, pp. 19567-19578, 2023, doi: 10.1109/ACCESS.2023.3249191.

[3] M. Saleh, M. Abbas and R. B. Le Jeannès, "FallAllD: An Open Dataset of Human Falls and Activities of Daily Living for Classical and Deep Learning Applications," in IEEE Sensors Journal, vol. 21, no. 2, pp. 1849-1858, 15 Jan.15, 2021, doi: 10.1109/JSEN.2020.3018335.

[4] J. Antonio Santoyo-Ramón, E. Casilari, and J. Manuel Cano-García, “A study of the influence of the sensor sampling frequency on the performance of wearable fall detectors,” *Measurement*, vol. 193, p. 110945, Apr. 2022, doi: 10.1016/j.measurement.2022.110945.

[5] R. Espinosa, H. Ponce, S. Gutiérrez, L. Martínez-Villaseñor, J. Brieva, and E. Moya-Albor, “A vision-based approach for fall detection using multiple cameras and convolutional neural networks: A case study using the UP-Fall detection dataset,” *Computers in Biology and Medicine*, vol. 115, p. 103520, Dec. 2019.

[6] M. Bastico, V. Ruiz Bejerano, and A. Belmonte-Hernández, “Simultaneous Real-Time Human Fall Detection and Reidentification Based on Multisensors Data,” in *Proceedings of the 15th International Conference on PErvasive Technologies Related to Assistive Environments*, in PETRA ’22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 365–370.

[7] J. Clemente, F. Li, M. Valero, and W. Song, “Smart Seismic Sensing for Indoor Fall Detection, Location, and Notification,” *IEEE Journal of Biomedical and Health Informatics*, vol. 24, no. 2, pp. 524–532, Feb. 2020, doi: 10.1109/JBHI.2019.2907498.

[8] M. Salman Khan, M. Yu, P. Feng, L. Wang, and J. Chambers, “An unsupervised acoustic fall detection system using source separation for sound interference suppression,” *Signal Processing*, vol. 110, pp. 199–210, May 2015, doi: 10.1016/j.sigpro.2014.08.021.

[9] T. Han, W. Kang, and G. Choi, “IR-UWB Sensor Based Fall Detection Method Using CNN Algorithm,” *Sensors*, vol. 20, no. 20, Art. no. 20, Jan. 2020, doi: 10.3390/s20205948.

[10] L. Martínez-Villaseñor, H. Ponce, J. Brieva, E. Moya-Albor, J. Núñez-Martínez, and C. Peñafort-Asturiano, “UP-Fall Detection Dataset: A Multimodal Approach,” *Sensors*, vol. 19, no. 9, p. 1988, Apr. 2019.

[11] J. Marques and P. Moreno, “Online Fall Detection Using Wrist Devices,” Sensors, vol. 23, no. 3, p. 1146, Jan. 2023, doi: 10.3390/s23031146.

[12] Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015.

[13] Pedregosa et al., “Scikit-learn: Machine Learning in Python,” JMLR 12, pp. 2825-2830, 2011.

[14] S. Li and C. Chiu, “A Smart Pillow for Health Sensing System Based on Temperature and Humidity Sensors,” Sensors, vol. 18, no. 11, 2018, doi: 10.3390/s18113664.

[15] S. Li, “Prediction of Body Temperature from Smart Pillow by Machine Learning,” in *2019 IEEE International Conference on Mechatronics and Automation (ICMA)*, Aug. 2019, pp. 421–426.